# Security Assessment Report

*Target URL: http://127.0.0.1:8000*

## Scan Metadata

| | |
|---|---|
| **Date of Scan:** | 2026-07-11 11:18:29 UTC |
| **Scan Duration:** | 10.35 seconds |
| **Total Findings:** | 1 |

## Vulnerability Severity Summary

| CRITICAL | HIGH | MEDIUM | LOW | INFO |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 |

## Executive Summary

Automated security audit was successfully completed. Review findings and remediation guidelines.

# Reconnaissance Details

| | |
|---|---|
| **Resolved Target IP:** | 127.0.0.1 |
| **Technologies Profile:** | Nginx, React |
| **Discovered Subdomains:** | api.127.0.0.1 |

# Detailed Security Findings

## 1. SQL Injection in /api/users - *CRITICAL Severity* [CVE-2023-SQLI]

**Scanner Source:** SQL Injection Scanner | **Confidence:** Low

**Description:**

SQL Injection occurs when untrusted input is directly concatenated into SQL query strings. An attacker can manipulate queries to extract data, modify tables, or execute database administrative operations.

**Remediation Guidance:**

1. Use Parameterized Queries: Replace dynamic SQL strings with prepared statements or ORM binding methods (e.g., SQLAlchemy parameters).
2. Apply Input Validation: Use schema validation (like Pydantic) to verify that parameters match expected formats.
3. Principle of Least Privilege: Ensure the database user has minimal necessary permissions.

**Technical Evidence:**

```
--- REQUEST ---
POST /api/users/profile
{
  "id": "1' OR (SELECT 1 FROM (SELECT(SLEEP(5)))a)--",
  "auth_token": "eyJhbGciOiJIUzI1NiI..."
}

--- RESPONSE ---
Response Time: 5012ms
HTTP/1.1 200 OK
Content-Type: application/json
```